Observabilidad en Spring Boot

Prometheus · Loki · Grafana

De "¿está caído?" a "¿por qué está lento, para quién y desde cuándo?"

Arquitectura de Software · Con resultados reales de una demo instrumentada

DEFINICIÓN

¿Qué es observabilidad?

La capacidad de entender el estado interno de un sistema a partir de las señales que emite, incluso ante preguntas que no anticipaste.

Viene de control

Término de la teoría de control (Kalman, 1960): un sistema es observable si sus salidas permiten inferir su estado.

En software

Las salidas son métricas, logs y trazas. El "estado" es: qué request, qué usuario, qué dependencia, qué versión.

La prueba

Si un problema nuevo te obliga a agregar un log y redesplegar para entenderlo, al sistema le falta observabilidad.

NO SON LO MISMO

Monitoreo vs. observabilidad

	Monitoreo	Observabilidad
Pregunta	¿Está funcionando?	¿Por qué se comporta así?
Tipo de problema	Conocidos ("known unknowns")	Nuevos ("unknown unknowns")
Artefacto	Checks, umbrales, alertas	Telemetría rica y consultable
Ejemplo	CPU > 80% → alerta	p99 de /pagos sube solo con la pasarela X
Relación	Es un subconjunto…	…de una práctica más amplia

Monitoreo te dice QUE algo falla. Observabilidad te deja averiguar POR QUÉ.

CONTEXTO

¿Cuándo invertir en observabilidad?

Sí, desde el día uno

- Microservicios o sistemas distribuidos
- Dependencias externas (pagos, APIs de terceros)
- Tienes SLOs o contratos de disponibilidad
- Varios equipos operan el mismo sistema
- Despliegues frecuentes (CI/CD)

Con mesura

- Monolito pequeño o prototipo: Actuator + logs estructurados alcanzan
- Scripts batch: métricas de éxito/duración del job
- Cuidado con el costo: la telemetría también se paga (almacenamiento, cardinalidad)
- Empieza por RED en los endpoints críticos

SEÑALES

Los tres pilares de la telemetría

Métricas

Números agregados en el tiempo.

¿Cuánto? ¿Qué tan rápido? ¿Qué porcentaje?

Baratas, ideales para alertar.

En la demo: Micrometer → Prometheus

Logs

Eventos discretos con contexto.

¿Qué pasó exactamente en este request?

Ricos pero caros de buscar si no son estructurados.

En la demo: Logback JSON → Loki

Trazas

El recorrido de un request entre servicios.

¿Dónde se fue el tiempo?

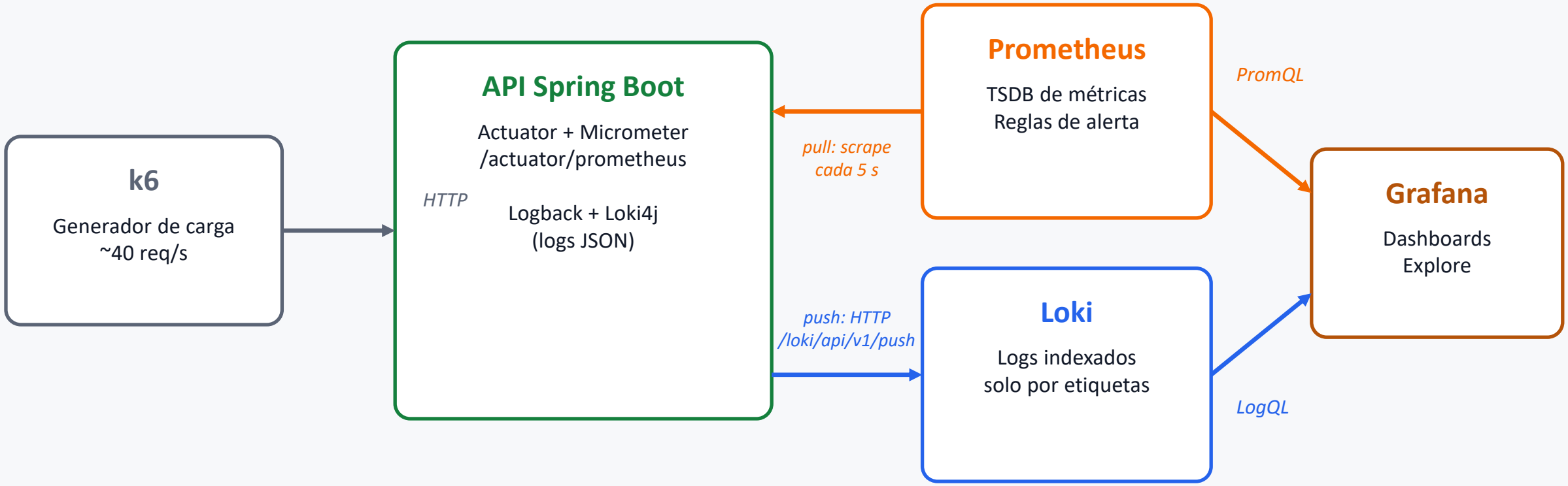
Clave en microservicios.

Siguiente paso: Micrometer Tracing + Tempo

El valor está en correlacionarlos: una métrica te lleva al log, el log a la traza.

CÓMO FLUYE LA TELEMETRÍA

Arquitectura: Spring → Prometheus / Loki → Grafana



Todo se levanta con un solo docker compose up: datasources y dashboard ya provisionados.

ESCENARIO

La demo: una tienda con dos problemas escondidos

Endpoint	Qué simula	Carga k6
`POST /api/pagos`	Pasarela externa: 90% rápida, 8% lenta, 2% muy lenta, ~4% timeouts	20 req/s
`GET /api/pedidos/n-mas-1`	JPA con relación LAZY: 1 consulta + 1 por cada pedido	5 req/s
`GET /api/pedidos/optimizado`	Mismo resultado con JOIN FETCH	5 req/s
`GET /api/catalogo`	Endpoint sano de referencia	10 req/s

7.204

requests en 4 minutos

40 req/s

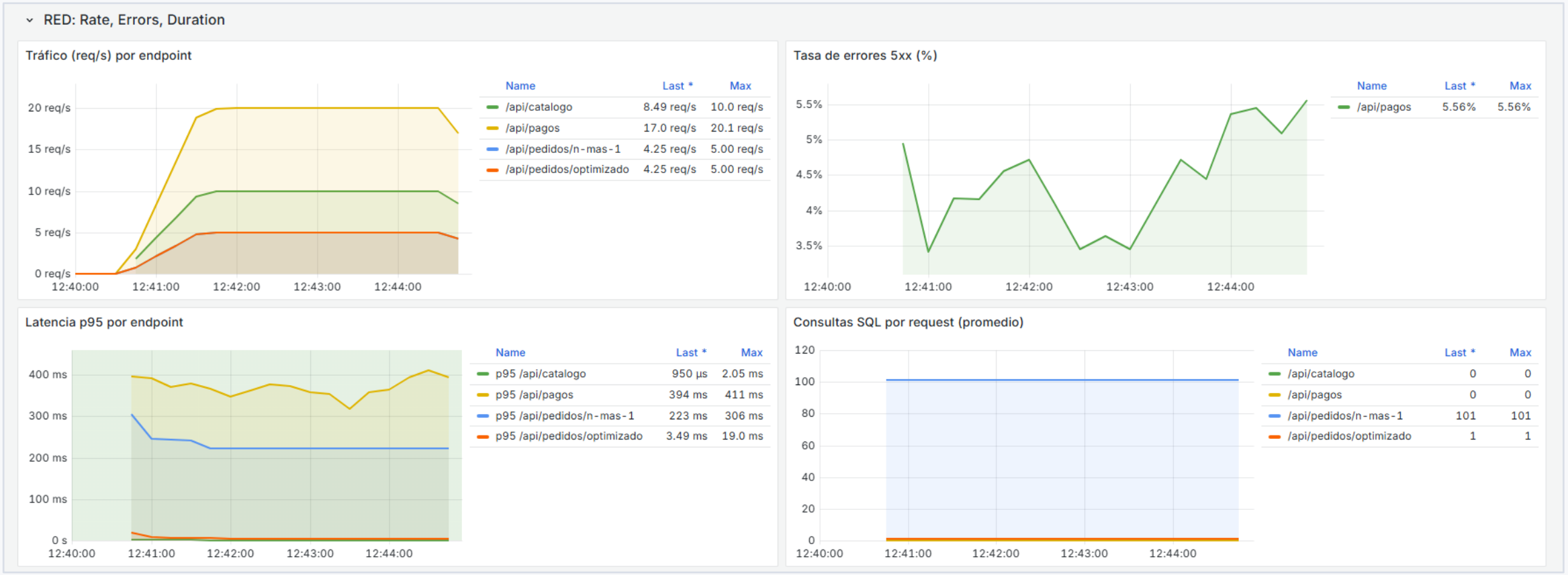
tráfico sostenido

100 pedidos × 3 ítems

H2 en memoria + 2 ms simulados de red a la BD

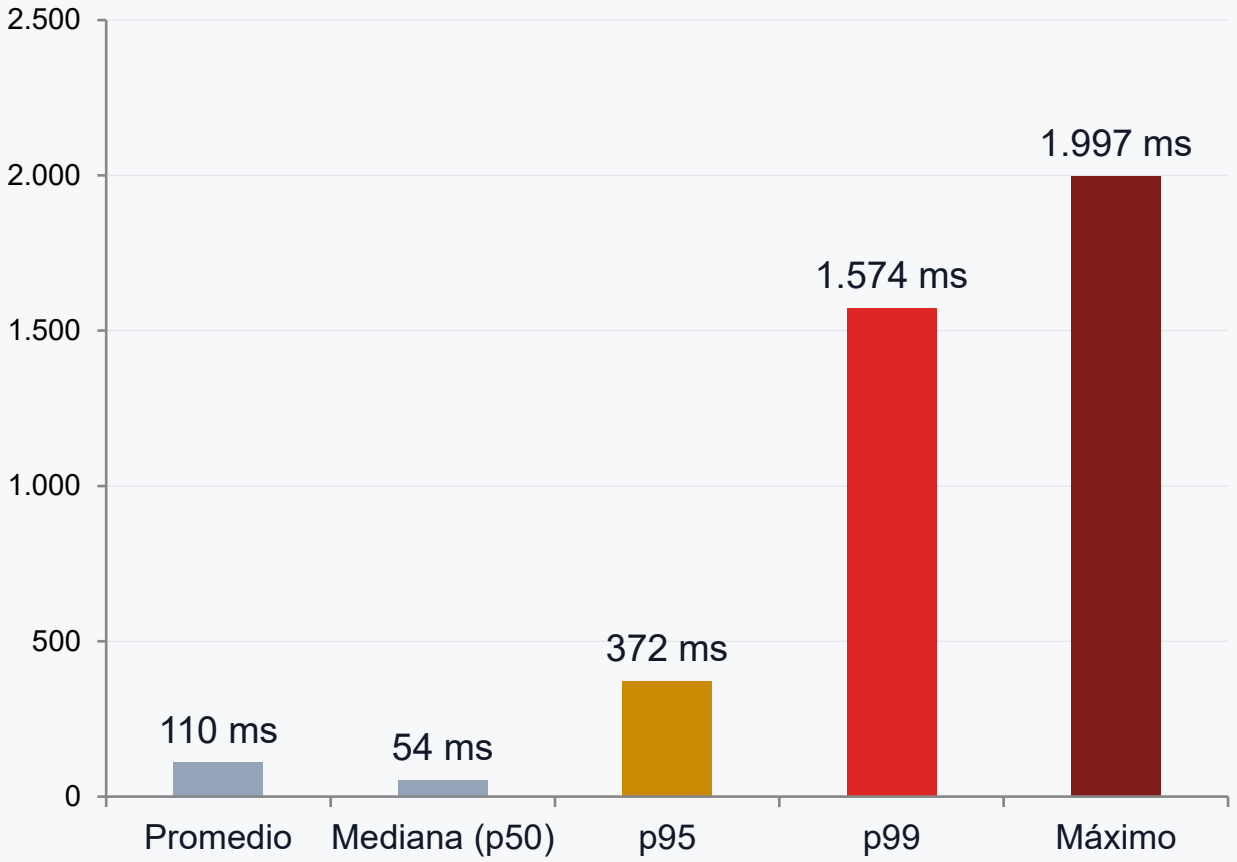
RESULTADO DE LA CARGA

El dashboard RED en Grafana (datos reales)



Tráfico · Errores 5xx · Latencia p95 · Consultas SQL por request — todo provisionado como código.

Cuello de botella 1: el promedio miente



Promedio 110 ms parece sano. Pero 1 de cada 100 pagos tarda **más de 1,5 s**. A 20 pagos por segundo, eso es un cliente frustrado cada 5 segundos.

PromQL

```
histogram_quantile(0.99,
  sum by (le, uri) (
    rate(http_server_requests_seconds_bucket
         {uri="/api/pagos"}[1m])))
```

k6 midió del lado del cliente p95 = 396 ms y p99 = 1.636 ms: coincide con Prometheus.

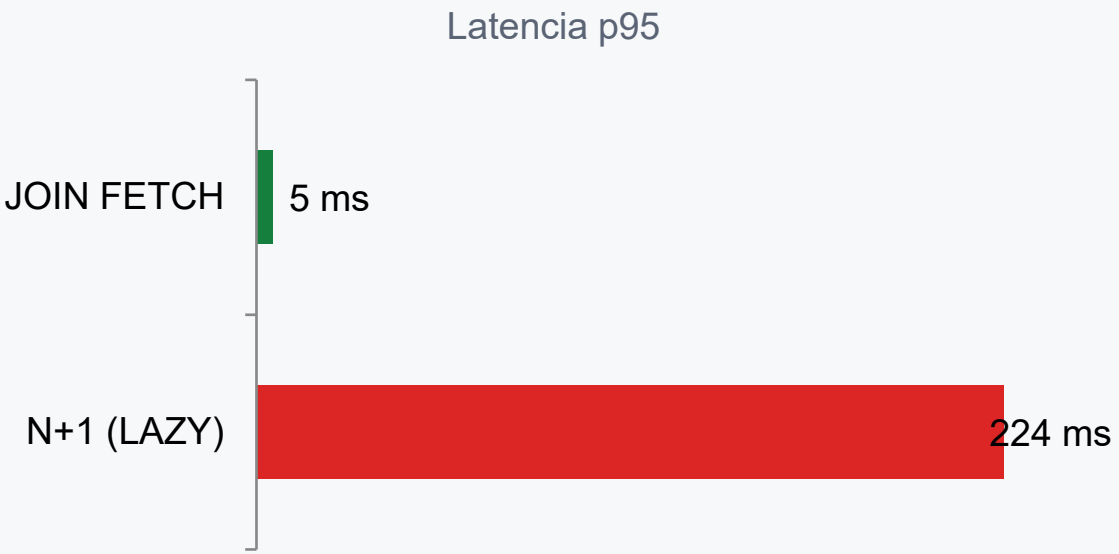
Cuello de botella 2: el problema N+1

101

consultas SQL por request (N+1)

1

consulta con JOIN FETCH



El log estructurado lo delata (LogQL)

```
{app="api-demo"}
  | json
  | mdc_sql_consultas > 50
```

```
"message":"request GET /api/pedidos/n-mas-1
  -> 200 en 217 ms (101 consultas SQL)",
"mdc_request_id":"20efa79a",
"mdc_sql_consultas":"101"
```

Arreglo: una línea en el repositorio

```
@Query("select distinct p from Pedido p
        left join fetch p.items")
```

≈ 45× más rápido con el mismo resultado

/API/PAGOS · ERRORES

De la alerta al log: la tasa de errores



4,4%

de los pagos fallan (144 de 3.601)

FIRING

alerta TasaErroresAlta (> 2% durante 1 min)

1. La métrica alerta (PromQL)

```
sum by (uri)(rate(http_server_requests_seconds_count
  {status=~"5.."}[2m]))
/ sum by (uri)(rate(http_server_requests_seconds_count[2m]))
> 0.02
```

2. El log explica (LogQL)

```
{app="api-demo", level="ERROR"}
```

```
"level":"ERROR",
"message":"Error procesando pago:
  La pasarela de pagos respondio timeout",
"mdc_request_id":"ef1851b4"
```

Loki: 9.511 INFO · 92 WARN (todos requests lentos de /pagos) · 213 ERROR

QUÉ MEDIR

Dos métodos para no perderse: RED y USE

RED · para servicios

Rate: requests por segundo
Errors: requests fallidos
Duration: distribución de latencia (p50/p95/p99)

Perspectiva del usuario. Tom Wilkie (Grafana Labs).

USE · para recursos

Utilization: % de tiempo ocupado
Saturation: trabajo en cola
Errors: errores del recurso

CPU, memoria, hilos, pool de conexiones. Brendan Gregg.

En la demo: RED marcó /pagos y /pedidos; USE mostró la JVM sana (heap ≤ 145 MiB de 5,6 GiB, máx. 5 de 50 hilos Tomcat, CPU < 6%). **Conclusión: el cuello no es de capacidad, es de diseño y de dependencias. Escalar servidores no lo habría arreglado.**

LO QUE HARÍA UN EQUIPO SENIOR

Buenas prácticas

1

Logs estructurados (JSON) con contexto

request_id, uri, duración, usuario anonimizado. Se consultan como datos.

2

Percentiles e histogramas, no promedios

Activa percentiles-histogram y define SLOs en p95 y p99.

3

Alertar por síntomas, investigar por causas

Errores y latencia que ve el usuario; la CPU va al dashboard, no al pager.

4

Etiquetas de baja cardinalidad

uri como plantilla (/pedidos/{id}), nunca el id. En Loki: pocas etiquetas.

5

Métricas de negocio

pagos aprobados/fallidos, carritos abandonados: lo que le importa al negocio.

6

Observabilidad como código

Dashboards, datasources y alertas versionados en Git (como esta demo).

LO QUE NO HAY QUE HACER

Errores comunes (antipatrones)

Explosión de cardinalidad

userId, requestId o email como etiqueta en Prometheus o Loki.

Dashboards que nadie mira

40 paneles sin pregunta detrás. Cada panel debe responder algo.

Fatiga de alertas

Alertas sin acción clara → el equipo las silencia → se pierde la importante.

Loguear todo (o nada)

DEBUG en producción, datos personales o contraseñas en logs.

Confiar en el promedio

Oculta la cola: 110 ms de promedio con un p99 de 1.574 ms en la demo.

Instrumentar después del incidente

La telemetría que no existía cuando pasó, no sirve para explicarlo.

20 MINUTOS · MANOS A LA OBRA

Taller guiado: ahora lo encuentran ustedes

0-4 min

Levantar

Codespace (o Docker local) y un solo comando: docker compose up

4-6 min

Carga

k6 genera tráfico 6 min mientras exploran

6-16 min

3 misiones

1. La cola de latencia (PromQL)
2. Cazar el N+1 (métrica + LogQL)
3. Del error al request_id (alerta + logs)

16-20 min

Reto

Crear una alerta de p99 > 1 s y verla disparar

Guía paso a paso: observabilidad-demo/TALLER.md (también en PDF). Nada que instalar: abren un GitHub Codespace desde el navegador.

Al final: preguntas y comparamos sus números con los de esta presentación.